

Real-Time Speaker Confidence Analyzer

Project Documentation

Yonas Tadesse

June 2026

Contents

1	Introduction	3
1.1	Overview	3
1.2	Motivation	3
1.3	Goals and Objectives	3
1.4	Related Work	4
1.5	Contributions	4
2	Problem Statement	5
3	Methodology	6
3.1	Approach	6
3.2	Pose Estimation	6
3.3	Confidence-Scoring Technique	6
3.4	Experimental Setup	7
3.5	Evaluation Metrics	7
4	Proposed Solution	9
4.1	Solution Summary	9
4.2	How the Components Work Together	9
5	System Architecture	11
5.1	High-Level Overview	11
5.2	Component Interaction	12
6	Implementation Details	13
6.1	Project Structure	13
6.2	Hardware	13
6.3	Software Stack	14
6.4	Key Implementation Specifics	14
6.5	Data Model	15
6.6	REST API	15
6.7	Running the System	15
7	Results	16
7.1	What the System Produces	16
7.2	Performance	16
7.3	Limitations	17

7.4	Future Work	17
8	Deployment	18
8.1	Current Deployment Model	18
8.2	Path to a Real-World Deployment	18
9	Conclusion	19
10	References	20

Chapter 1

Introduction

1.1 Overview

Public speaking is one of the most common sources of performance anxiety, and one of the hardest skills to self-correct, because a speaker cannot easily observe their own body language while they are delivering it. Recording a practice session and reviewing it afterward helps, but the feedback is unstructured: the speaker has to watch the whole video back and guess, by eye, whether their posture, gestures, and orientation came across as confident.

This project, the **Real-Time Speaker Confidence Analyzer**, addresses that gap with a system that watches a speaker through a webcam, estimates their body pose in real time using a YOLO pose-estimation model, converts that pose into a quantitative confidence score, displays that score live while the speaker is practicing, and logs every session to a database so that progress can be reviewed afterward in a web dashboard.

1.2 Motivation

The motivation for the project is twofold. First, objective, real-time feedback on body language is largely absent from how people practice speeches today — most practice is either unobserved or reviewed informally by a friend or by re-watching a recording. Second, modern real-time pose-estimation models (the YOLO-Pose family, built on the YOLO object-detection architecture) make it computationally feasible to extract a person’s skeletal keypoints from a webcam feed on ordinary laptop hardware, in real time, without specialized sensors (no motion-capture suit, no depth camera, no multi-camera rig). This made it possible to build a lightweight, fully local tool: a webcam and a laptop are the only hardware requirements.

1.3 Goals and Objectives

The project set out to:

- Detect a speaker’s body pose in real time from a single webcam using a pose-estimation model (YOLO11n-pose).
- Translate raw pose keypoints into a small set of interpretable confidence-related metrics (posture, orientation, gesture activity, and so on) rather than a single opaque score.

- Combine those metrics into one overall confidence score that updates continuously during a session.
- Give the speaker live visual feedback (an on-screen skeleton overlay, the live score, and short textual coaching cues) while they are speaking.
- Persist every session so that confidence trends can be tracked over time rather than only seen in the moment.
- Provide a web-based dashboard for reviewing past sessions: summary statistics, a confidence timeline, a per-metric breakdown, and the best/worst moments of a session.

1.4 Related Work

The system sits at the intersection of two areas. The first is real-time human pose estimation, a well-established computer-vision problem with mature open solutions such as OpenPose, MediaPipe Pose, and the pose-estimation variants of the YOLO object-detection family (YOLO-Pose), the last of which this project builds on directly. These models estimate a fixed set of body keypoints (commonly the 17-point COCO keypoint format) from a single RGB image, which is exactly the input this project needs.

The second area is automated public-speaking and presentation-skills feedback, an application space that has grown alongside the maturity of pose- and speech-analysis models. Existing approaches in this space typically focus on one of two signals: vocal delivery (pace, filler words, tone) or visual delivery (posture, gesture, eye contact). This project focuses on the visual-delivery side, using YOLO-Pose as the perception layer and a custom geometric scoring layer on top of it, and leaves vocal-delivery analysis as noted future work.

1.5 Contributions

The contribution of this project is an open, end-to-end pipeline — rather than a single model or algorithm — that connects perception, scoring, live feedback, storage, and analytics into one working system:

- A custom heuristic scoring layer that converts raw YOLO-Pose keypoints into six interpretable sub-metrics (visibility, posture straightness, body orientation, hand openness, gesture activity, head position) and a single weighted confidence score, computed without any additional model training.
- A real-time feedback loop built on OpenCV that overlays the skeleton, the live score, color-coded coaching text, a session timer, and a per-metric sidebar directly on the video feed.
- A persistence layer (SQLite) that logs a data point roughly every 500 ms during a session and stores session-level summary statistics.
- A Flask REST API and a Next.js/React analytics dashboard that turn the logged data into a timeline chart, a radar chart of the six metrics, a confidence-distribution chart, and a “key moments” view highlighting the best and worst points of a session.

Chapter 2

Problem Statement

Speakers preparing for a talk, presentation, or interview have very few ways to get **objective, real-time** feedback on the body-language side of their delivery. The available options today fall short in specific ways:

- **Unassisted practice** (rehearsing alone or in front of a mirror) gives no quantitative signal at all — the speaker is both the performer and the only judge, in the moment, of their own posture and gestures.
- **Self-review of a recording** after the fact requires the speaker to manually watch back a video and subjectively judge their own body language, which is time-consuming, inconsistent between sessions, and does not produce a trackable metric over time.
- **Human coaching** is effective but is not always available, is not real-time during independent practice, and does not scale to frequent, low-stakes rehearsal.

At the same time, the underlying perception problem — estimating where a person’s limbs, head, and torso are in a video frame — is now solvable in real time on commodity hardware thanks to pretrained pose-estimation models. What is missing is the layer that turns those raw keypoints into something a speaker can act on immediately: a single, continuously updating confidence indicator, paired with the more granular metrics that explain it, available live during practice and preserved afterward for comparison across sessions.

The problem this project addresses is therefore: **how to convert real-time body-pose keypoints from a single webcam into an immediate, interpretable, and persisted confidence signal that a speaker can use both during and after a practice session**, without requiring specialized hardware, manual annotation, or a human observer.

Chapter 3

Methodology

3.1 Approach

The system follows a perceive-score-feedback-store pipeline. Each video frame from the webcam is passed through a pretrained pose-estimation model to obtain body keypoints; those keypoints are converted into six geometric, rule-based metrics; the metrics are combined into a single weighted confidence score; the score and metrics are rendered back onto the video in real time; and a sample of the score stream is periodically written to a database for later analysis.

3.2 Pose Estimation

Body pose is estimated with **YOLO11n-pose**, the nano-sized pose-estimation variant of Ultralytics’ YOLO11 model, loaded through the **ultralytics** Python package. For each frame the model returns, for the detected person, 17 keypoints in the standard COCO format (nose, eyes, ears, shoulders, elbows, wrists, hips, knees, ankles), each with (x, y) image coordinates and a per-keypoint confidence score. Inference is run with a detection confidence threshold of 0.45 on every captured frame.

3.3 Confidence-Scoring Technique

Rather than training a model to predict “confidence” directly (for which no labeled dataset of speaker confidence exists), the project uses a **geometric, rule-based scoring technique** derived from the keypoint coordinates:

- **Visibility** — the fraction of the 17 keypoints whose detection confidence exceeds 0.4, expressed as a percentage. If visibility falls below 65%, the system short-circuits to a low confidence estimate, since the remaining metrics depend on keypoints that are not reliably visible.
- **Posture straightness** — the angle between the shoulder-to-hip (“spine”) vector and the vertical axis. A near-vertical spine (small tilt angle) scores close to 100; a slouched or leaning posture is penalized proportionally to the tilt angle.
- **Body orientation** — the ratio of shoulder width to torso length (shoulder-to-hip distance). A speaker facing the camera presents a wide shoulder span relative to torso length; a speaker turned sideways presents a narrower one, which lowers the score.

- **Hand openness** — derived from the distance between the two wrists and the distance from the midpoint of the wrists to the body’s center. Wrists held close together or close to the torso (crossed-arm or clasped-hand postures) reduce the score; open, away-from-body hand positions increase it.
- **Gesture activity** — the frame-to-frame displacement of both wrists, measured against the wrist position from 10 frames earlier (a rolling history of up to 45 frames is kept for this purpose). Larger recent wrist movement increases the score, rewarding natural hand gesturing over static arms.
- **Head position** — the vertical offset of the nose keypoint relative to the shoulder line. A head held level with or above the shoulder midline scores higher; a head dropped well below it (looking down) is penalized.

Each of the six metrics is computed on a 0–100 scale, and the overall confidence score is a fixed weighted average of all six (weights: visibility 15%, posture 20%, orientation 20%, hand openness 18%, gesture activity 15%, head position 12%), clipped to the 15–98 range so the displayed score never reads as a false 0% or 100%.

3.4 Experimental Setup

- **Capture:** a single webcam stream captured via OpenCV at 1280×720.
- **Session length:** a fixed 4-minute (240-second) timed session, intended to match the length of a typical short speech or presentation rehearsal.
- **Sampling rate:** one data point (overall score plus all six metrics) is logged approximately every 500 ms, balancing the granularity of the resulting timeline against the size of the stored session.
- **Frame-extraction utility:** a helper script (`src/preprocess/extract_frames.py`) was used to sample still frames out of test/reference videos (one frame roughly every 30 frames) for manual, visual spot-checking of how the pose model and scoring metrics behaved on footage outside of live webcam testing — for example a sample talk video used during development is present in `data/frames/`.
- **Model weights:** the project uses the stock, publicly pretrained `yolo11n-pose.pt` weights as distributed by Ultralytics; no project-specific fine-tuning of the pose model has been performed (this is listed under future work).

3.5 Evaluation Metrics

Two complementary kinds of evaluation are used:

1. **The metrics themselves as the evaluation output** — for every session the system records the average, maximum, and minimum overall confidence score, the time spent in the high (greater than 70), medium (40–70), and low (under 40) confidence bands, and the session-average of each of the six sub-metrics. These are the quantities the dashboard visualizes.
2. **Qualitative validation** — because there is no labeled ground-truth dataset of “true” speaker confidence to benchmark against, correctness was checked qualitatively: by observing the live OpenCV overlay (skeleton, bounding box, color-coded score) while deliberately performing confident versus slouched/closed body language, and by confirming that the reported metrics moved in the expected direction (for example, that crossing one’s arms visibly reduced

the hand-openness and overall score). This limitation — no formal, human-rated accuracy benchmark — is discussed further in the Results section.

Chapter 4

Proposed Solution

4.1 Solution Summary

The proposed solution is a three-tier, fully local system that turns a webcam feed into a real-time confidence score during a speaking session, and into a browsable analytics history afterward. It is composed of six cooperating components:

1. **Pose Capture & Inference** — captures webcam frames and runs YOLO11n-pose on each one to obtain 17 body keypoints per frame.
2. **Confidence Scoring Engine** — computes the six geometric sub-metrics described in the Methodology and combines them into one weighted overall confidence score per frame.
3. **Real-Time Feedback Layer** — an OpenCV-rendered overlay showing the live confidence score, a color-coded coaching message, a skeleton drawn over the detected body, a bounding box, a countdown timer, and a sidebar of the six live sub-metric bars.
4. **Session Persistence** — at roughly 500 ms intervals, the current score and metrics are appended to an in-memory list; at the end of the session, summary statistics are computed and the full session (summary plus every logged data point) is written to a local SQLite database.
5. **REST API** — a small Flask application reads from the SQLite database and exposes the session list and per-session detail (including all data points) as JSON over HTTP, so that the data is consumable by any client.
6. **Analytics Dashboard** — a Next.js/React web application consumes the REST API and renders a session list plus, for each session, a confidence-over-time line chart, a six-axis radar chart of the metric averages, a high/medium/low confidence-distribution doughnut chart, and a “key moments” panel highlighting the single best and worst points in the session.

4.2 How the Components Work Together

During a live session, components 1–4 run continuously in a single Python process: each frame flows from capture, through pose inference, through scoring, onto the screen as feedback, and (every 500 ms) into the session’s data log. When the timer reaches zero or the user stops the session early, the accumulated data points and computed summary statistics are written to SQLite in one batch.

Components 5 and 6 form a separate, decoupled read path: the Flask API and the Next.js dashboard do not need to be running during a live session at all (the analyzer writes directly to the SQLite

file) — they are only needed afterward, when the speaker wants to review their session history. This separation means the latency-sensitive, real-time part of the system (capture → inference → scoring → display) never depends on network calls or a second process being available.

Chapter 5

System Architecture

5.1 High-Level Overview

The system is organized as three independent, loosely coupled tiers, each of which can be started and run on its own:

- a **real-time analyzer** (Python process: capture, inference, scoring, live display, and writes to the database),
- a **data store** (a single SQLite database file), and
- a **web tier** (a Flask REST API reading from the database, and a Next.js dashboard consuming that API).

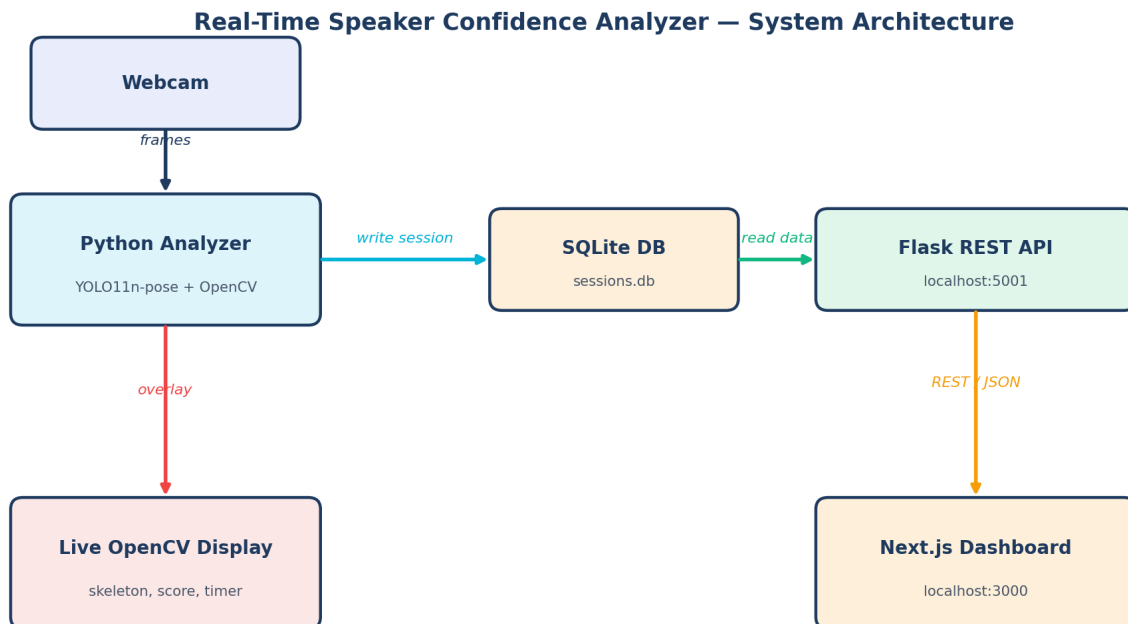


Figure 5.1: System architecture

5.2 Component Interaction

- The **Webcam** supplies raw frames to the **Python Analyzer** process.
- The **Python Analyzer** runs YOLO11n-pose inference and the scoring logic on each frame, renders the **Live OpenCV Display** (skeleton overlay, score, timer, metric bars) in the same process, and at session end writes the full session record to the **SQLite DB**.
- The **Flask REST API** reads from the same SQLite DB file on request and serves session data as JSON.
- The **Next.js Dashboard** calls the Flask API over HTTP (`localhost:5001`) to fetch the session list and per-session detail, and renders the analytics views in the browser (`localhost:3000`).

Because the analyzer writes directly to the database file rather than through the API, the real-time capture path has no runtime dependency on the API or dashboard processes — those two only need to be running when the speaker wants to *review* past sessions, not while a new session is being recorded.

Chapter 6

Implementation Details

6.1 Project Structure

```
speakers_confidence/
|-- requirements.txt          # Python dependencies
|-- Makefile                 # the three commands needed to run the system
|-- src/
|   |-- speech_confidence_analyzer.py  # main analyzer: capture, inference, scoring, display
|   |-- yolo11n-pose.pt           # pretrained YOLO11n-pose weights
|   |-- api/
|   |   |-- server.py             # Flask REST API
|   |-- models/
|   |   |-- database.py           # SQLite schema + read/write helpers
|   |-- preprocess/
|   |   |-- extract_frames.py      # utility: sample frames from test videos
|-- dashboard/                # Next.js analytics dashboard
|   |-- src/
|   |   |-- app/
|   |   |   |-- page.js           # session list page
|   |   |   |-- session/[id]/page.js  # session detail page
|   |   |-- components/
|   |   |   |-- SessionCard.js
|   |   |   |-- SummaryCards.js
|   |   |   |-- ConfidenceTimeline.js
|   |   |   |-- MetricRadar.js
|   |   |   |-- ConfidenceDistribution.js
|   |   |-- KeyMoments.js
```

6.2 Hardware

The system targets ordinary consumer hardware: a laptop or desktop with a built-in or USB webcam, and a CPU (or modest GPU) capable of running the nano-sized YOLO11n-pose model in real time. The nano model size was specifically chosen, instead of a larger YOLO-Pose variant, so

that inference stays fast enough for live feedback without requiring a dedicated GPU.

6.3 Software Stack

Analyzer / backend (Python):

Library	Role
ultralytics	loads and runs YOLO11n-pose inference
opencv-python	webcam capture, skeleton/overlay rendering, live display window
numpy	vector math for all six geometric metrics
flask / flask-cors	REST API and cross-origin access for the dashboard
sqlite3 (standard library)	session and data-point persistence

Dashboard (JavaScript):

Library	Role
next (14.x, App Router)	dashboard application framework
react / react-dom (18.x)	UI components
chart.js / react-chartjs-2	line, radar, and doughnut charts

6.4 Key Implementation Specifics

- **Skeleton drawing:** the 17 COCO keypoints are connected with 16 fixed limb pairs (e.g., shoulder–elbow, elbow–wrist, hip–knee, knee–ankle); a limb is only drawn if both of its endpoint keypoints have a detection confidence above 0.4.
- **Score-to-color mapping:** scores of 72 or above render green, 48–71 render yellow, and below 48 render red, both in the live OpenCV overlay and (conceptually) in the dashboard’s distribution chart.
- **Fallback behavior:** if fewer than 17 keypoints are returned at all, the system returns a flat default score of 40 with all sub-metrics at 0, rather than crashing or producing a misleading high score.
- **Low-visibility short-circuit:** if visibility (the fraction of confidently detected keypoints) is below 65%, the function returns early with a reduced score, since the geometric metrics that follow (posture, orientation, etc.) are not reliable when most keypoints are missing.
- **Gesture-activity window:** a deque of up to 45 past frames of keypoints is kept; gesture activity compares the current wrist positions to the wrist positions 10 frames earlier, so it reflects recent movement rather than a single-frame snapshot.
- **Logging cadence:** a data point is appended to the session’s in-memory log whenever at least 500 ms have elapsed since the previous logged point, decoupling the logging rate from the (much higher) camera frame rate.

6.5 Data Model

The SQLite database (`src/data/sessions.db`) has two tables:

sessions — one row per recorded session: `id` (text, primary key), `name`, `date` (ISO 8601), `duration` (seconds), `avg_confidence`, `max_confidence`, `min_confidence`, `time_high` / `time_medium` / `time_low` (seconds spent in each confidence band), and the session-average of each of the six sub-metrics (`avg_visibility`, `avg_posture_straightness`, `avg_body_orientation`, `avg_hand_openness`, `avg_gesture_activity`, `avg_head_position`).

data_points — one row per logged sample within a session (roughly every 500 ms): `id` (auto-increment primary key), `session_id` (foreign key into `sessions`, cascading delete), `timestamp` (milliseconds from session start), `confidence`, and the six instantaneous sub-metric values.

6.6 REST API

Method	Endpoint	Purpose
GET	<code>/api/health</code>	health check
GET	<code>/api/sessions</code>	list all sessions (summary rows only)
GET	<code>/api/sessions/<id></code>	full detail for one session, including all data points
DELETE	<code>/api/sessions/<id></code>	delete a session and its data points

The API runs on `localhost:5001` with CORS enabled so the dashboard (on `localhost:3000`) can call it directly from the browser.

6.7 Running the System

The project is run as three separate processes, as captured in the Makefile:

```
# Terminal 1 - Flask API
cd src && python -m api.server

# Terminal 2 - Dashboard
cd dashboard && npm run dev

# Terminal 3 - Record a session
cd src && python speech_confidence_analyzer.py
```

Chapter 7

Results

7.1 What the System Produces

For every recorded session, the system produces both a real-time read-out and a persisted analytical record. During the session, the speaker sees their live confidence score, the six contributing sub-metric bars, and a short coaching message that changes based on score (a “low confidence” warning below 42, general posture/orientation/hand cues between 42 and 65, and a positive confirmation above 65).

After the session, the dashboard surfaces, per session: average / maximum / minimum confidence and total duration (summary cards); a confidence-over-time line chart; a six-axis radar chart of the session’s average sub-metrics; a doughnut chart of time spent in the high/medium/low confidence bands; and a “key moments” panel calling out the single highest- and lowest-scoring points in the session and when they occurred.

As an illustrative example, a representative practice session produced an average confidence of 68.2%, a maximum of 89.1%, a minimum of 34.5%, over a 4-minute duration, with roughly 50% of the session spent in the high band, 33% in the medium band, and 17% in the low band — the kind of breakdown the dashboard is designed to make immediately visible to the speaker. At an individual-frame level, a “good confident posture” reading corresponded to high visibility and posture scores (95 and 80) together with moderate-to-high orientation and head-position scores, producing an overall score around 78%; lower overall scores were reliably associated with lower hand-openness and gesture-activity values, consistent with the intent of those two metrics.

7.2 Performance

Because YOLO11n-pose is the smallest pose-estimation variant in the YOLO11 family, the analyzer is able to run pose inference and render the live overlay on a standard laptop without a dedicated GPU, which was a hard requirement for the system to feel responsive as *live* feedback rather than a delayed or choppy one.

7.3 Limitations

- **No formal accuracy benchmark.** The confidence score and its six sub-metrics are hand-designed geometric heuristics, not learned from labeled examples of “confident” versus “unconfident” speakers. Their behavior has been validated qualitatively (by observing that deliberate posture/gesture changes move the score in the expected direction) rather than against a human-rated ground truth, so the absolute scores should be read as a consistent relative signal rather than a calibrated, externally validated measure of confidence.
- **Single-person assumption.** The scoring logic always reads the first detected person (`result.keypoints.xy[0]`); the system is not designed for multi-person scenes.
- **Sensitivity to camera setup.** Because several metrics (orientation, posture, hand openness) are derived from 2D keypoint geometry, results depend on the camera being reasonably front-facing and the speaker’s full upper body being in frame; unusual camera angles or partial framing can distort the geometric assumptions behind the metrics.
- **Untuned weights and thresholds.** The metric weights (15/20/20/18/15/12%) and the various distance/angle thresholds inside `analyze_posture()` were chosen by hand rather than fit to data.
- **Visual-only signal.** The system currently analyzes body language exclusively; it does not analyze speech content, tone, pace, or filler words.

7.4 Future Work

In line with the limitations above, the planned directions for the project are: fine-tuning the pose model (or, more likely, the scoring weights) using real speech footage such as TED talks, so that the metric weighting reflects empirically observed confidence cues rather than hand-tuned defaults; and extending the system to analyze the speaker’s voice as well — tone, pace, and filler-word usage — so that the overall feedback covers both the visual and the vocal side of a delivery.

Chapter 8

Deployment

8.1 Current Deployment Model

Today the system is designed to run entirely on a single local machine, started as three independent processes (see *Running the System* above): the Flask API on `localhost:5001`, the Next.js dashboard on `localhost:3000`, and the analyzer process with direct access to the machine’s webcam. There is no installer or orchestration script beyond the three commands recorded in the `Makefile`; all three are started manually in separate terminals.

8.2 Path to a Real-World Deployment

A few characteristics of the system shape how it would be deployed more broadly:

- **The analyzer must run close to the camera.** Because it needs low-latency access to a live webcam feed for real-time feedback, the pose-capture-and-scoring component is inherently a local, client-side piece of software — it is not something that can be moved to a remote server without reintroducing the camera-streaming latency the local design was meant to avoid. For wider distribution, this component would most naturally be packaged as a small desktop application (for example, with PyInstaller) so that a non-technical user could run it without a Python environment.
- **The data and web tiers can be centralized.** The Flask API, the SQLite database, and the Next.js dashboard do not need to run on the same machine as the analyzer. For a single user practicing on one machine, keeping everything local (as it is today) is simplest. For a setting where a speaker’s history should be available from multiple devices, or where a coach reviews several speakers’ sessions, the API and database would move to a small server, with SQLite likely replaced by a multi-user database such as PostgreSQL, and the dashboard deployed as a standard static/SSR Next.js site (for example, on a platform such as Vercel) pointed at that remote API.
- **Production hardening would be needed before any networked deployment.** The current Flask server runs in debug mode and with a wide-open CORS policy, both of which are appropriate for local development but not for an internet-facing deployment; moving to a production WSGI server (e.g., gunicorn) behind HTTPS, restricting CORS to the dashboard’s actual origin, and adding authentication (since session data is personal) would all be prerequisites for deploying the API and dashboard beyond a single user’s own machine.

Chapter 9

Conclusion

This project built a working, end-to-end system that turns a single webcam feed into real-time, quantified feedback on a speaker’s body language, and into a persisted, browsable history of that feedback over time. Using YOLO11n-pose for real-time keypoint estimation, a six-metric weighted scoring layer for turning those keypoints into an interpretable confidence score, an OpenCV-based live overlay for in-the-moment feedback, and a SQLite-backed Flask API plus Next.js dashboard for after-the-fact analysis, the system demonstrates that meaningful, real-time speaker feedback is achievable on ordinary consumer hardware, without specialized sensors or a trained “confidence” model.

The main implication of this work is practical rather than theoretical: it shows that a lightweight, hand-designed scoring layer on top of an off-the-shelf pose-estimation model is enough to produce a usable, real-time practice tool, and that the harder, more valuable next step is not better pose estimation (which is already mature) but better-validated scoring — ideally informed by real speech footage and, eventually, by the speaker’s voice as well as their body. The system’s own limitations, particularly the absence of a human-validated accuracy benchmark for its confidence score, mark the clearest direction for that next step.

Chapter 10

References

1. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
2. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., & Rabinovich, A. (2015). *Going Deeper with Convolutions*. IEEE Conference on Computer Vision and Pattern Recognition (CVPR). (GoogLeNet / Inception architecture, an architectural ancestor of the YOLO backbone family.)
3. Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., & Dollár, P. (2014). *Microsoft COCO: Common Objects in Context*. European Conference on Computer Vision (ECCV). (Source of the 17-keypoint human pose annotation format used by YOLO-Pose.)
4. Ultralytics. *YOLO11 / YOLO-Pose Documentation*. <https://docs.ultralytics.com>
5. Ultralytics. *Ultralytics YOLO (GitHub repository)*. <https://github.com/ultralytics/ultralytics>
6. Bradski, G. (2000). *The OpenCV Library*. Dr. Dobb's Journal of Software Tools.
7. Flask Documentation. <https://flask.palletsprojects.com>
8. Next.js Documentation. <https://nextjs.org/docs>
9. SQLite Documentation. <https://www.sqlite.org>